



Student Name: Bhavya
Branch: BE-CSE(AIT)
Semester: 4
Subject Name: Database Management System

UID: 24BAI70791
Section/Group: 24AIT_KRG2
Date of Performance: 27/02/2026
Subject Code: 24CSH-298

WORKSHEET 7

AIM: To design and implement a **materialized view** and to **compare and analyze execution time** and performance differences between **simple views**, **complex views**, and **materialized views**, thereby understanding their impact on query optimization and system performance.

2. Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

3. Objective

To create **simple views**, **complex views**, and **materialized views**, and to evaluate their performance by comparing query execution time for each, highlighting the advantages of materialized views in enterprise-level applications.

4. Procedure of the Experiment

1. Open PostgreSQL (pgAdmin) / Oracle SQL Developer and connect to the database.
2. Create the required tables (**Employee**, **Department**).
3. Insert sample data into the tables.

4. Verify the data using SELECT queries.
5. Create a **simple view** using a single table.
6. Execute the simple view to check output.
7. Create a **complex view** using JOIN and aggregation.
8. Create a **materialized view** to store computed results.
9. Execute all views and analyze performance using **EXPLAIN ANALYZE**.
10. Compare execution time and conclude that materialized view performs better.

PROBLEM STATEMENT:

In large-scale enterprise systems, frequent execution of complex queries can significantly affect performance. Optimized mechanisms are required to improve query response time without compromising data consistency.

TABLE STRUCTURE:

```
[SQL Worksheet]*  ▶  ⚙  🔍  📄  ⌕  Aa  🗑  
1  CREATE TABLE emp_table (  
2      emp_id NUMBER PRIMARY KEY,  
3      emp_name VARCHAR2(50),  
4      dept_id NUMBER,  
5      salary NUMBER  
6  );
```

```
SQL> CREATE TABLE emp_table (  
      emp_id NUMBER PRIMARY KEY,  
      emp_name VARCHAR2(50),  
      dept_id NUMBER,...  
Show more...
```

Table EMP_TABLE created.

Elapsed: 00:00:00.014

```
CREATE TABLE department (  
    dept_id NUMBER PRIMARY KEY,  
    dept_name VARCHAR2(50)  
);
```

```
SQL> CREATE TABLE department (  
    dept_id NUMBER PRIMARY KEY,  
    dept_name VARCHAR2(50)  
)
```

Table DEPARTMENT created.

Elapsed: 00:00:00.015

```
13  INSERT INTO department VALUES (1, 'HR');  
14  INSERT INTO department VALUES (2, 'IT');  
15  
16  INSERT INTO emp_table VALUES (101, 'John', 1, 3000);  
17  INSERT INTO emp_table VALUES (102, 'Alice', 2, 5000);  
18  INSERT INTO emp_table VALUES (103, 'Bob', 2, 4000);  
19  INSERT INTO emp_table VALUES (104, 'Emma', 1, 3500);  
20  
21  COMMIT;
```

1 row inserted.

Elapsed: 00:00:00.001

```
SQL> COMMIT
```

Commit complete.

Elapsed: 00:00:00.001

1. PROBLEM STATEMENT – Simple View

Create a simple view based on a single table.

PROGRAM:

```
CREATE VIEW emp_view AS
SELECT emp_id, emp_name, salary
FROM emp_table;
```

OUTPUT:

```
SQL> CREATE VIEW emp_view AS
      SELECT emp_id, emp_name, salary
      FROM emp_table
```

View EMP_VIEW created.

Elapsed: 00:00:00.013

2. PROBLEM STATEMENT –Complex View

Create a complex view using join and aggregation.

PROGRAM:

```
CREATE VIEW emp_dept_view AS
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM emp_table e
JOIN department d ON e.dept_id = d.dept_id
GROUP BY d.dept_name;
```

OUTPUT:

```
SQL> CREATE VIEW emp_dept_view AS
      SELECT d.dept_name, AVG(e.salary) AS avg_salary
      FROM emp_table e
      JOIN department d ON e.dept_id = d.dept_id...
Show more...
```

View EMP_DEPT_VIEW created.

Elapsed: 00:00:00.009

3. PROBLEM STATEMENT – Materialized View

Create a materialized view to store precomputed results.



```
CREATE MATERIALIZED VIEW emp_mv  
BUILD IMMEDIATE  
REFRESH COMPLETE  
AS  
SELECT d.dept_name, AVG(e.salary) AS avg_salary  
FROM emp_table e  
JOIN department d ON e.dept_id = d.dept_id  
GROUP BY d.dept_name;
```

OUTPUT:

```
SQL> CREATE MATERIALIZED VIEW emp_mv  
BUILD IMMEDIATE  
REFRESH COMPLETE  
AS...  
Show more...
```

Materialized view EMP_MV created.

Elapsed: 00:00:00.119

REFRESH MATERIALIZED VIEW:

```
EXEC DBMS_MVIEW.REFRESH('EMP_MV');
```

OUTPUT:

```
SQL> EXEC DBMS_MVIEW.REFRESH('EMP_MV')
```

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.142

PERFORMANCE ANALYSIS:

QUERY ON SIMPLE VIEW:

```
SELECT * FROM emp_view;
```

OUTPUT:

	EMP_ID	EMP_NAME	SALARY
1	101	John	3000
2	102	Alice	5000
3	103	Bob	4000
4	104	Emma	3500

QUERY ON COMPLEX VIEW:

```
SELECT * FROM emp_dept_view;
```

OUTPUT:

	DEPT_NAME	AVG_SALARY
1	HR	3250
2	IT	4500

QUERY ON MATERIALIZED VIEW:

```
SELECT * FROM emp_mv;
```

OUTPUT:

	DEPT_NAME	AVG_SALARY
1	HR	3250
2	IT	4500



LEARNING OUTCOMES:

After completing this experiment, the learner will be able to:

- Understand the concept and working of **materialized views** in a database system.
- Differentiate between **simple views**, **complex views**, and **materialized views**.
- Create and refresh materialized views in PostgreSQL.
- Measure and compare **query execution time** for different types of views.
- Analyze performance benefits of materialized views in data-intensive applications.
- Apply materialized view concepts in real-world company scenarios such as **SanDisk, JTG, and PayPal**.